Création d'un NDIS en Python : Théorie et Scripts

Partie 1 : Théorie sur les NDIS et Scapy

Un NDIS (Network Intrusion Detection System) est un système crucial pour la sécurité des réseaux. Il **surveille le trafic réseau pour détecter toute activité suspecte** et joue un rôle clé dans la protection contre les cyberattaques. Un NDIS analyse en permanence le trafic entrant et sortant afin d'identifier des comportements anormaux. Il utilise diverses méthodes pour identifier les menaces potentielles, comme la comparaison de signatures, l'analyse heuristique et l'apprentissage automatique. Lorsqu'une activité suspecte est détectée, le NDIS envoie des alertes aux administrateurs réseau et fournit des rapports détaillés sur les incidents de sécurité.

Scapy est une bibliothèque Python puissante et interactive conçue pour la manipulation de paquets réseau. Elle offre une flexibilité et des fonctionnalités avancées qui en font un outil idéal pour la création d'un NDIS. Scapy permet aux utilisateurs de **créer, manipuler, envoyer, capturer et analyser des paquets réseau de manière très détaillée**.

Avantages d'utiliser Scapy pour la création d'un NDIS:

- **Construction précise des paquets:** Scapy offre un contrôle granulaire sur chaque aspect du paquet, permettant la création de paquets personnalisés pour tester les réseaux et les protocoles.
- **Analyse et décodage des protocoles:** Scapy peut décoder des paquets de différents protocoles, même ceux qui sont moins courants, ce qui est essentiel pour une analyse approfondie du trafic réseau.
- **Capture de trafic en temps réel:** Scapy peut capturer des paquets en temps réel sur le réseau, permettant l'analyse du trafic en direct pour la détection d'anomalies.

Partie 2 : Exemples de scripts Python pour la détection d'intrusions

Les sources fournissent des exemples concrets de scripts Python utilisant Scapy pour détecter différents types d'attaques réseau.

Détection d'un scan SYN

Ce script utilise la fonction `sniff` de Scapy pour capturer les paquets TCP et une fonction personnalisée (`detecter_scan_syn`) pour analyser chaque paquet et identifier les tentatives de scan SYN.

```
from scapy.all import sniff, TCP, IP
from datetime import date

def detecter_scan_syn(packet):
    if IP in packet and TCP in packet and packet[TCP].flags == 'S':
        src_ip = packet[IP].src
        print(f"Scan SYN détecté de {src_ip}")
        info = {
            'ipSource': packet[IP].src,
            'ipDest': packet[IP].dst,
            'portSource': packet[IP].sport,
```

```
            'portDest': packet[IP].dport,
            'horodatage': date.today(),
            'activite': 'Scan SYN',
            'description': f'Tentative de SCAN SYN détectée sur le service
à destination de {packet[IP].src}'
        }
        return info

sniff(filter="tcp", prn=detecter_scan_syn)
```

Ce script recherche les paquets TCP qui ont le drapeau SYN activé sans réponse ACK, ce qui est un indicateur d'un scan SYN. L'adresse IP source du paquet suspect est affichée, et un dictionnaire contenant des informations détaillées sur l'événement est créé.

Détection d'une attaque DDOS

Ce script utilise Scapy pour surveiller les tentatives de connexion sur des ports spécifiques et identifie les pics d'activité qui pourraient indiquer une attaque DDOS.

```
from scapy.all import sniff, IP, TCP
from collections import Counter
from time import time

# Liste des ports à surveiller
ports_surveilles =

# Dictionnaire pour compter le nombre de tentatives de connexion par IP
source et par port
tentatives_connexion = Counter()

def compter_tentatives(packet):
    if IP in packet and TCP in packet:
        src_ip = packet[IP].src
        dst_port = packet[TCP].dport
        if dst_port in ports_surveilles:
            tentatives_connexion[(src_ip, dst_port)] += 1
            if tentatives_connexion[(src_ip, dst_port)] >= 10:
                return create_ddos_alert(src_ip, packet[IP].dst, dst_port)

def create_ddos_alert(src_ip, dst_ip, dst_port):
    alert_time = time()
    alert_info = {
        'ipSource': src_ip,
        'ipDest': dst_ip,
        'portSource': "source nombreuse",
        'portDest': dst_port,
        'horodatage': alert_time,
        'activite': 'Attaque DDOS',
        'description': f'Tentative de DDOS détectée sur le service à
destination de {dst_ip}'
    }
    return alert_info

# Bien que non présent dans le code que vous avez fourni, la fonction
`sniff` serait typiquement utilisée comme suit pour démarrer la capture et
l'analyse des paquets :
# sniff(filter="tcp", prn=compter_tentatives)
```

Le script utilise un compteur pour suivre le nombre de tentatives de connexion par adresse IP source et par port de destination. Si le nombre de tentatives dépasse un seuil prédéfini, une alerte DDOS est générée.

Détection d'une attaque par force brute

Ce script identifie les tentatives répétées de connexion à partir d'une même adresse IP sur des ports spécifiques, ce qui est caractéristique d'une attaque par force brute.

```
from scapy.all import sniff, IP, TCP
from collections import defaultdict
import time

# Dictionnaire pour stocker les tentatives de connexion par adresse IP
nbrConnection = defaultdict(list)

# Limite de tentatives pour considérer une attaque par force brute
limite = 10

# Fenêtre de temps en secondes pour analyser les tentatives de connexion
time_window = 60

# Liste des ports à surveiller
ports_surveilles =

def detect_bruteforce(packet):
    if IP in packet and TCP in packet:
        ipSource = packet[IP].src
        ipDest = packet[IP].dst
        portSource = packet[TCP].sport
        portDest = packet[TCP].dport
        date = time.time()

        # Enregistrer la tentative de connexion
        nbrConnection[ipSource].append(date)

        # Nettoyer les tentatives plus anciennes que la fenêtre de temps
        nbrConnection[ipSource] = [t for t in nbrConnection[ipSource] if
date - t <= time_window]

        # Détecter une attaque par force brute
        if len(nbrConnection[ipSource]) == limite:
            timestamp = time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(date))
            if portDest in ports_surveilles:
                nbrConnection[ipSource] = []  # Réinitialiser après
détection
                return alert_bruteforce(ipSource, ipDest, portSource,
portDest, timestamp)
    return None

def alert_bruteforce(ipSource, ipDest, portSource, portDest, date):
    info = {
        'ipSource': ipSource,
        'ipDest': ipDest,
        'portSource': portSource,
        'portDest': portDest,
        'horodatage': date,
        'activite': 'BruteForce',
```

```
        'description': f'Tentative de force brute détectée sur le service à
destination du port {portDest}'
    }
    return info

sniff(filter="tcp", prn=detect_bruteforce)
```

Le script enregistre les tentatives de connexion pour chaque adresse IP et vérifie si le nombre de tentatives dépasse un seuil défini dans une fenêtre de temps donnée. Si une attaque est détectée, une alerte est générée.

Conclusion

Ces exemples illustrent comment Scapy peut être utilisé pour créer un NDIS efficace en Python. La flexibilité de Scapy permet de développer des modules de détection pour diverses attaques réseau. Il est possible de créer un programme principal qui intègre ces modules et utilise des threads pour effectuer les détections en parallèle.

Informations supplémentaires non issues des sources

Il est important de noter que les exemples fournis dans les sources sont des exemples simplifiés. Un NDIS complet nécessiterait des fonctionnalités supplémentaires, comme l'intégration avec des systèmes SIEM, la mise à jour régulière des signatures et des règles de détection, et l'utilisation de l'apprentissage automatique pour améliorer la détection des menaces inconnues. De plus, une configuration minutieuse du NDIS est essentielle pour minimiser les faux positifs et les faux négatifs.